**EEL 4742C: Embedded Systems**
**Name:**Ryan Ramdihal
**Lab 6:** Universal Asynchronous Receiver and Transmitter (UART)

## Introduction:

In this lab, we implement the UART communication on the MSP430FR6989 microcontroller, alongside its timer functions and LED operations. The objective is to establish a communication channel utilizing UART while still demonstrating timer interrupts in sync with LED toggling. UART is initialized to operate at a baud rate of 9600 to transmit integers and strings, while also using input to toggle the green LED. Then being able to showcase modifications to the UART configuration.

## 6.1 Transmitting Data over UART

The objective of this code is to implement UART communication along with the MSP430fr6989's timer functions and LEDs. After configuring the LEDs and the ACLK to a 1 second delay, The UART is also initialized to operate at the 9600 baud rate with 8 bit data and 1 stop bit. The code continuously iterates ASCII characters representing numbers 0-9 over the UART with newlines and return characters for formatting. It reads the input characters from the UART of the computer and toggles the green LED based on the received data. If '1' is pressed on the keyboard the green LED will turn on, and if '2' is pressed on the keyboard it will turn off. Additionally, the red LED is toggled every 1 second interrupt from TimerA, to show the program is running. Overall, this code demonstrates UART communication on the microcontroller.

## 6.1 Code:

```
#include <msp430fr6989.h>
#define FLAGS UCA1IFG // Contains the transmit & receive flags
#define RXFLAG UCRXIFG // Receive flag, 0:no new data; 1:new data received
#define TXFLAG UCTXIFG // Transmit flag, 0:busy, 1:ready
#define TXBUFFER UCA1TXBUF // Transmit buffer
#define RXBUFFER UCA1RXBUF // Receive buffer //reading clears Rx flag
#define redLED BIT0 // Red LED at P1.0
#define greenLED BIT7 // Green LED at P9.7
#define BUT1 BIT1 // Button S1 at P1.1
void config_ACLK_to_32KHz_crystal();
void Initialize_UART();
void uart_write_char(unsigned char ch);
unsigned char uart_read_char();
int main()
{
   WDTCTL = WDTPW | WDTHOLD;   // stop watchdog timer
   PM5CTL0 &=  ~LOCKLPM5; // Enable the GPIO pins
   // Configure and initialize LEDs
   P1DIR |= redLED; // Direct pin as output
   P9DIR |= greenLED; // Direct pin as output
   P1OUT &=  ~redLED; // Turn LED Off
   P9OUT &=  ~greenLED; // Turn LED Off
```

```
    config_ACLK_to_32KHz_crystal();
    // Configure Channel 0 for up mode with interrupts
    TA0CCR0 = 32767; // 1 second @ 32 KHz
    // Timer_A: ACLK, div by 1, up mode, clear TAR
    TA0CTL = TASSEL_1 | ID_0 | MC_1 | TACLR;
    // Configure pins to backchannel UART
    // Pins: (UCA1TXD / P3.4) (UCA1RXD / P3.5)
    // (P3SEL1=00, P3SEL0=11) (P2DIR=xx)
    P3SEL1 &=  ~(BIT4|BIT5);
    P3SEL0 |= (BIT4|BIT5);
    Initialize_UART();
    unsigned char digit = 48; //ascii of 0
    unsigned char input = 0;
    unsigned char data;
    while(1){
        uart_write_char(digit); //Write 0
        uart_write_char(10); //Write /n ascii of new line
        uart_write_char(13); //Write /r ascii of return
        _delay_cycles(500000); //Delay
        digit++; //iterate

        if (digit == 58) //when it hits 9 reset to 0
            digit = 48;
        input = uart_read_char(); //Read input
        if (input != 0){
            data = input; //If input was not null, set data
        }
        if (data == 49) // if 1 is pressed
            P9OUT |= greenLED; //If data was 1, set LED
        else if (data == 50)
            P9OUT &= ~greenLED; //If data was 0, disable LED // if 2 is pressed
        if ((TA0CTL & TAIFG) == TAIFG){ // clock cycle reset
                TA0CTL &= ~ TAIFG; //Clear flag
                P1OUT ^= redLED; //Toggle LED
      }
    }
}
// Configure UART to the popular configuration
// 9600 baud, 8-bit data, LSB first, no parity bits, 1 stop bit
// no flow control, oversampling reception
// Clock: SMCLK @ 1 MHz (1,000,000 Hz)
void Initialize_UART(){
    // Configure pins to UART functionality
    P3SEL1 &=  ~(BIT4|BIT5);
    P3SEL0 |= (BIT4|BIT5);
    // Main configuration register
    UCA1CTLW0 = UCSWRST; // Engage reset; change all the fields to zero
    // Most fields in this register, when set to zero, correspond to the
    // popular configuration
    UCA1CTLW0 |= UCSSEL_2; // Set clock to SMCLK
    // Configure the clock dividers and modulators (and enable oversampling)
    UCA1BRW = 6; // divider
    // Modulators: UCBRF = 8 = 1000 --> UCBRF3 (bit #3)
    // UCBRS = 0x20 = 0010 0000 = UCBRS5 (bit #5)
```

```
    UCA1MCTLW = UCBRF3 | UCBRS5 | UCOS16;
    // Exit the reset state
    UCA1CTLW0 &=  ~UCSWRST;
    }
// The function returns the byte; if none received, returns null character
unsigned char uart_read_char(){
    unsigned char temp;
    // Return null character (ASCII=0) if no byte was received
    if((FLAGS & RXFLAG) == 0)
        return 0;
    // Otherwise, copy the received byte (this clears the flag) and return it
    else
        temp = RXBUFFER;
        return temp;
}
void uart_write_char(unsigned char data){
    // Wait for any ongoing transmission to complete
    while ( (FLAGS & TXFLAG)==0 ) {}
    // Copy the byte to the transmit buffer
    TXBUFFER = data; // Tx flag goes to 0 and Tx begins!
    return;
}
void config_ACLK_to_32KHz_crystal() {
    // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
    // Reroute pins to LFXIN/LFXOUT functionality
    PJSEL1 &= ~BIT4;
    PJSEL0 |= BIT4;
    // Wait until the oscillator fault flags remain cleared
    CSCTL0 = CSKEY; // Unlock CS registers
    do {
        CSCTL5 &= ~LFXTOFFG; // Local fault flag
        SFRIFG1 &= ~OFIFG; // Global fault flag
    } while((CSCTL5 & LFXTOFFG) != 0);
    CSCTL0_H = 0; // Lock CS registers
    return;
}
```

## 6.2 Transmitting Integers & Strings over UART

The next task is to create two functions for transmitting data over UART: one for transmitting unsigned 16-bit integers and another for transmitting strings. The function for transmitting integers takes an unsigned integer as input and converts it into its ASCII representation. It calculates the number of digits in the integer, parses each digit, converts it to its corresponding ASCII value, and then sends each ASCII character sequentially over UART. This function allows the program to transmit numerical data as a sequence of ASCII characters. The function for transmitting strings iterates through a character array until it reaches the null terminator, sending each character over UART, effectively transmitting the entire string.

## 6.2 Code:

```
#include <msp430fr6989.h>
#include <stdio.h>
#define FLAGS UCA1IFG // Contains the transmit & receive flags
#define RXFLAG UCRXIFG // Receive flag
#define TXFLAG UCTXIFG // Transmit flag
#define TXBUFFER UCA1TXBUF // Transmit buffer
#define RXBUFFER UCA1RXBUF // Receive buffer
#define redLED BIT0 // Red LED at P1.0
#define greenLED BIT7 // Green LED at P9.7
#define BUT1 BIT1 // Button S1 at P1.1
void config_ACLK_to_32KHz_crystal();
void Initialize_UART();
void uart_write_char(unsigned char ch);
unsigned char uart_read_char();
void uart_write_uint16(unsigned int n);
void uart_write_string(char str[]);
unsigned int ascii[10] = {48, 49, 50, 51, 52, 53, 54, 55, 56, 57}; //0-9
int main()
{
   WDTCTL = WDTPW | WDTHOLD;   // stop watchdog timer
   PM5CTL0 &= ~LOCKLPM5; // Enable the GPIO pins
   // Configure and initialize LEDs
   P1DIR |= redLED; // Direct pin as output
   P9DIR |= greenLED; // Direct pin as output
   P1OUT &= ~redLED; // Turn LED Off
   P9OUT &= ~greenLED; // Turn LED Off
   config_ACLK_to_32KHz_crystal();
   // Configure Channel 0 for up mode with interrupts
   TA0CCR0 = 32767; // 1 second @ 32 KHz
   // Timer_A: ACLK, div by 1, up mode, clear TAR
   TA0CTL = TASSEL_1 | ID_0 | MC_1 | TACLR;
   // Configure pins to backchannel UART
   // Pins: (UCA1TXD / P3.4) (UCA1RXD / P3.5)
   // (P3SEL1=00, P3SEL0=11) (P2DIR=xx)
   P3SEL1 &= ~(BIT4|BIT5);
   P3SEL0 |= (BIT4|BIT5);
   Initialize_UART();
   unsigned char input;
   unsigned char data;
   unsigned int n = 65535;//0-65535
   char str[] = "testing if this works";
   //Write number
   uart_write_uint16(n);
   uart_write_string(str);
   while(1){
      _delay_cycles(420000); //Delay
      input = uart_read_char(); //Read input
      if (input != 0){
         data = input; //If input was not null, set data
      }
      if (data == 49)
         P9OUT |= greenLED; //If data was 1, set LED
      else if (data == 50)
         P9OUT &= ~greenLED; //If data was 0, disable LED
```

```
        if ((TA0CTL & TAIFG) == TAIFG){ //reset clock cycle
            TA0CTL &= ~ TAIFG; //Clear flag
            P1OUT ^= redLED; //Toggle LED
        }
    }
}
void uart_write_uint16(unsigned int n){
    int i;
        unsigned int numSize = 1; // Initialize numSize to 1
        unsigned int num = n; // Create a copy of n to avoid modifying the original value
        // Count number of digits
        while (num /= 10) {
            numSize++;
        }
        unsigned int digits[5]; // Number will not have more than 5 digits
        // Parse the numbers digit, place into declared array
        for (i = numSize - 1; i >= 0; i--) {
            digits[i] = n % 10;
            n = n / 10;
        }
        // Write each digit over UART
        for (i = 0; i < numSize; i++) {
            uart_write_char(ascii[digits[i]]);
        }
        uart_write_char(10); // Write /n
        uart_write_char(13); // Write /r
    }
void uart_write_string(char str[]){
    int i = 0;
    while(str[i] != '\0'){
        uart_write_char(str[i]);//write in array letter by letter
        i++;
    }
    uart_write_char(10); //Write /n
    uart_write_char(13); //Write /r
}
//Get the number of digits in a certain number
unsigned int get_numSize(unsigned int n){
    //count number of digits
    unsigned int size=1;
    while (n/=10) size++;
    return size;
}
// Configure UART to the popular configuration
// 9600 baud, 8-bit data, LSB first, no parity bits, 1 stop bit
// no flow control, oversampling reception
// Clock: SMCLK @ 1 MHz (1,000,000 Hz)
void Initialize_UART(){
    // Configure pins to UART functionality
    P3SEL1 &=  ~(BIT4|BIT5);
    P3SEL0 |= (BIT4|BIT5);
    // Main configuration register
    UCA1CTLW0 = UCSWRST; // Engage reset; change all the fields to zero
    // Most fields in this register, when set to zero, correspond to the
```

```
    // popular configuration
    UCA1CTLW0 |= UCSSEL_2; // Set clock to SMCLK
    // Configure the clock dividers and modulators (and enable oversampling)
    UCA1BRW = 6; // divider
    // Modulators: UCBRF = 8 = 1000 --> UCBRF3 (bit #3)
    // UCBRS = 0x20 = 0010 0000 = UCBRS5 (bit #5)
    UCA1MCTLW = UCBRF3 | UCBRS5 | UCOS16;
    // Exit the reset state
    UCA1CTLW0 &=  ~UCSWRST;
    }
// The function returns the byte; if none received, returns null character
unsigned char uart_read_char(){
    unsigned char temp;
    // Return null character (ASCII=0) if no byte was received
    if( (FLAGS & RXFLAG) == 0)
        return 0;
    // Otherwise, copy the received byte (this clears the flag) and return it
    temp = RXBUFFER;
    return temp;
    }
void uart_write_char(unsigned char ch){
    // Wait for any ongoing transmission to complete
    while ( (FLAGS & TXFLAG)==0 ) {}
    // Copy the byte to the transmit buffer
    TXBUFFER = ch; // Tx flag goes to 0 and Tx begins!
    return;
    }
void config_ACLK_to_32KHz_crystal() {
    // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
    // Reroute pins to LFXIN/LFXOUT functionality
    PJSEL1 &= ~BIT4;
    PJSEL0 |= BIT4;
    // Wait until the oscillator fault flags remain cleared
    CSCTL0 = CSKEY; // Unlock CS registers
    do {
        CSCTL5 &= ~LFXTOFFG; // Local fault flag
        SFRIFG1 &= ~OFIFG; // Global fault flag
    } while((CSCTL5 & LFXTOFFG) != 0);
    CSCTL0_H = 0; // Lock CS registers
    return;
    }
```

## 6.3 Modifying the UART Configuration

The next task involves adjusting the UART configuration to utilize ACLK instead of SMCLK and establishing a baud rate of 4800. To switch the clock to utilize ACLK, we set the UCSSEL_1 in the UCA1CTLW0 register .After referring to the family guide for the appropriate dividers and modulators we find that only UCBRSx and UCBR are relevant for this modification. The divider is still 6. UCBRSx are the upper 8 bits (15-8) making UCA1MCTLW equal to 0xEE00. Checking the function by configuring our terminal and running the program. This adjustment ensures that the UART module operates reliably at 4800 baud using ACLK as the clock source.

## 6.3 Code:

```
#include <msp430fr6989.h>
#include <stdio.h>
#define FLAGS UCA1IFG // Contains the transmit & receive flags
#define RXFLAG UCRXIFG // Receive flag
#define TXFLAG UCTXIFG // Transmit flag
#define TXBUFFER UCA1TXBUF // Transmit buffer
#define RXBUFFER UCA1RXBUF // Receive buffer
#define redLED BIT0 // Red LED at P1.0
#define greenLED BIT7 // Green LED at P9.7
#define BUT1 BIT1 // Button S1 at P1.1
void config_ACLK_to_32KHz_crystal();
void Initialize_UART();
void uart_write_char(unsigned char ch);
unsigned char uart_read_char();
void uart_write_uint16(unsigned int n);
void uart_write_string(char str[]);
unsigned int ascii[10] = {48, 49, 50, 51, 52, 53, 54, 55, 56, 57}; //0-9
int main()
{
   WDTCTL = WDTPW | WDTHOLD;   // stop watchdog timer
   PM5CTL0 &= ~LOCKLPM5; // Enable the GPIO pins
   // Configure and initialize LEDs
   P1DIR |= redLED; // Direct pin as output
   P9DIR |= greenLED; // Direct pin as output
   P1OUT &= ~redLED; // Turn LED Off
   P9OUT &= ~greenLED; // Turn LED Off
   config_ACLK_to_32KHz_crystal();
   // Configure Channel 0 for up mode with interrupts
   TA0CCR0 = 32767; // 1 second @ 32 KHz
   // Timer_A: ACLK, div by 1, up mode, clear TAR
   TA0CTL = TASSEL_1 | ID_0 | MC_1 | TACLR;
   // Configure pins to backchannel UART
   // Pins: (UCA1TXD / P3.4) (UCA1RXD / P3.5)
   // (P3SEL1=00, P3SEL0=11) (P2DIR=xx)
   P3SEL1 &= ~(BIT4|BIT5);
   P3SEL0 |= (BIT4|BIT5);
   Initialize_UART_2();
   unsigned char input;
   unsigned char data;
   unsigned int n = 65535;//0-65535
   char str[] = "testing if this works";
   //Write number
   uart_write_uint16(n);
   uart_write_string(str);
   while(1){
      _delay_cycles(420000); //Delay
      input = uart_read_char(); //Read input
      if (input != 0){
         data = input; //If input was not null, set data
      }
      if (data == 49)
         P9OUT |= greenLED; //If data was 1, set LED
```

```
        else if (data == 50)
            P9OUT &= ~greenLED; //If data was 0, disable LED
        if ((TA0CTL & TAIFG) == TAIFG){ //reset clock cycle
              TA0CTL &= ~ TAIFG; //Clear flag
              P1OUT ^= redLED; //Toggle LED
       }
    }
}
void uart_write_uint16(unsigned int n){
    int i;
        unsigned int numSize = 1; // Initialize numSize to 1
        unsigned int num = n; // Create a copy of n to avoid modifying the original value
        // Count number of digits
        while (num /= 10) {
            numSize++;
        }
        unsigned int digits[5]; // Number will not have more than 5 digits
        // Parse the numbers digit, place into declared array
        for (i = numSize - 1; i >= 0; i--) {
            digits[i] = n % 10;
            n = n / 10;
        }
        // Write each digit over UART
        for (i = 0; i < numSize; i++) {
            uart_write_char(ascii[digits[i]]);
        }
        uart_write_char(10); // Write /n
        uart_write_char(13); // Write /r
    }
void uart_write_string(char str[]){
    int i = 0;
    while(str[i] != '\0'){
        uart_write_char(str[i]);//write in array letter by letter
        i++;
    }
    uart_write_char(10); //Write /n
    uart_write_char(13); //Write /r
}
//Get the number of digits in a certain number
unsigned int get_numSize(unsigned int n){
    //count number of digits
    unsigned int size=1;
    while (n/=10) size++;
    return size;
}
// Configure UART to the popular configuration
// 9600 baud, 8-bit data, LSB first, no parity bits, 1 stop bit
// no flow control, oversampling reception
// Clock: SMCLK @ 1 MHz (1,000,000 Hz)
void Initialize_UART(){
    // Configure pins to UART functionality
    P3SEL1 &=  ~(BIT4|BIT5);
    P3SEL0 |= (BIT4|BIT5);
    // Main configuration register
```

```
    UCA1CTLW0 = UCSWRST; // Engage reset; change all the fields to zero
    // Most fields in this register, when set to zero, correspond to the
    // popular configuration
    UCA1CTLW0 |= UCSSEL_2; // Set clock to SMCLK
    // Configure the clock dividers and modulators (and enable oversampling)
    UCA1BRW = 6; // divider
    // Modulators: UCBRF = 8 = 1000 --> UCBRF3 (bit #3)
    // UCBRS = 0x20 = 0010 0000 = UCBRS5 (bit #5)
    UCA1MCTLW = UCBRF3 | UCBRS5 | UCOS16;
    // Exit the reset state
    UCA1CTLW0 &=  ~UCSWRST;
    }
// The function returns the byte; if none received, returns null character
unsigned char uart_read_char(){
    unsigned char temp;
    // Return null character (ASCII=0) if no byte was received
    if( (FLAGS & RXFLAG) == 0)
        return 0;
    // Otherwise, copy the received byte (this clears the flag) and return it
    temp = RXBUFFER;
    return temp;
    }
void uart_write_char(unsigned char ch){
    // Wait for any ongoing transmission to complete
    while ( (FLAGS & TXFLAG)==0 ) {}
    // Copy the byte to the transmit buffer
    TXBUFFER = ch; // Tx flag goes to 0 and Tx begins!
    return;
    }
void config_ACLK_to_32KHz_crystal() {
    // By default, ACLK runs on LFMODCLK at 5MHz/128 = 39 KHz
    // Reroute pins to LFXIN/LFXOUT functionality
    PJSEL1 &= ~BIT4;
    PJSEL0 |= BIT4;
    // Wait until the oscillator fault flags remain cleared
    CSCTL0 = CSKEY; // Unlock CS registers
    do {
        CSCTL5 &= ~LFXTOFFG; // Local fault flag
        SFRIFG1 &= ~OFIFG; // Global fault flag
    } while((CSCTL5 & LFXTOFFG) != 0);
    CSCTL0_H = 0; // Lock CS registers
    return;
    }
void Initialize_UART_2(){
// Configure pins to UART functionality
P3SEL1 &=  ~(BIT4|BIT5);
P3SEL0 |= (BIT4|BIT5);
// Main configuration register
UCA1CTLW0 = UCSWRST; // Engage reset; change all the fields to zero
UCA1CTLW0 |= UCSSEL_1; // Set clock to ACLK
// Configure the clock dividers and modulators
UCA1BRW = 6; // divider
// UCBRS = 0xEE = 1110 1110 = (bits 1, 2, 3, 5, 6, 7)
UCA1MCTLW = 0xEE00; //upper bits 15-8 check page 779*786 of family guide
```

```
// Exit the reset state
UCA1CTLW0 &= ~UCSWRST;
}
```

## Conclusion:

This lab demonstrates the integration of UART communication with the MSP430fr6989's timer functions and LED operations. By configuring UART to transmit ASCII characters representing numbers and implementing functionality to toggle LEDs based on received UART data, the microcontroller showcases its ability to communicate tasks. With the flexibility to use different clocks and baud rates to ensure UART can handle practical applications for embedded systems.

## Student Q&A

1. What's the difference between UART and eUSCI?
eUSCI is the communication module that implements UART's serial communication of transmitting and receiving. eUSCI has 2 channels that use different communication protocols, one of them being UART.
2. What is the backchannel UART?
An interface of the microcontroller that is routed to the USB connector of the PC.
3. What's the function of the two lines of code that have P3SEL1 and P3SEL0?
To transmit and receive data between external devices and the microcontroller.
4. The microcontroller has a clock of 1,000,000 Hz and we want to setup a UART connection at 9600 baud. How do we obtain a clock rate of 9600 Hz? Explain the approach at a high level.
100000 Hz/ 9600Hz = 104.16, modulator mixing cycles between 1MHz/104 and 1MHz/105, to get the average frequency of 9600 Hz.
5. A UART transmitter is transmitting data at at 1200 baud. What is receiver's clock frequency if oversampling is not used?
1200 Hz

6. A UART transmitter is transmitting data at at 1200 baud. What is receiver's clock frequency if oversampling is used? What's the benefit of oversampling?
16 * 1200Hz = 19200Hz. The benefit of oversampling is improved accuracy of the data, reducing timing errors by recovering data in a bit period.